

Practical No. 4

Aim: Develop a program to implement doubly linked lists and circular linked lists, including insertion and deletion operations.

Source-Code:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* prev;
    Node* next;
};
Node* head = NULL;
void insert(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
    cout << "Inserted: " << value << endl;
}
void deleteNode(int value) {
    Node* temp = head;
    while (temp != NULL && temp->data
!= value) {
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "Value not found!" << endl;
        return;
    }
    if (temp == head) {
        head = temp->next;
        if (head != NULL) {
            head->prev = NULL;
        }
    } else {
        temp->prev->next = temp->next;
        if (temp->next != NULL) {
            temp->next->prev = temp->prev;
        }
        delete temp;
        cout << "Deleted: " << value << endl;
    }
}
void display() {
    Node* temp = head;
    cout << "Doubly Linked List: ";
    while (temp != NULL) {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
int main() {
    insert(11);
    insert(22);
    insert(33);
    insert(44);
    insert(55);
    display();

    deleteNode(44);

    display();

    return 0;
}
```

Output:

Inserted: 11

Inserted: 22

Inserted: 33

Inserted: 44

Inserted: 55

Doubly Linked List: 11 <-> 22 <-> 33 <-> 44 <-> 55 <-> NULL

Deleted: 44

Doubly Linked List: 11 <-> 22 <-> 33 <-> 55 <-> NULL